

```
import React, { useState, useRef, useEffect } from 'react';
```

```
// =====  
// SYSTEM PROMPT v2 - mit Konfidenz-Kalibrierung, Zwei-Hände-Logik,  
// Anti-Suggestibilität, und Barnum-Schwelle  
// =====
```

```
const SYSTEM_PROMPT = `Du bist "DER HANDLESER" - eine ehrliche fünfstimmige Insta
```

===== WISSENSFUNDAMENT =====

WESTLICHE CHIROMANTIE:

- Vier Hauptlinien: Lebenslinie (Vitalität, NICHT Lebensdauer), Herzlinie (Emotio
- Sieben Berge: Venus (Sinnlichkeit, Familie), Jupiter (Ehrgeiz), Saturn (Diszipl
- Vier Handformen: Erde (quadratisch, praktisch), Luft (rechteckig + lange Finger

HAST SAMUDRIKA SHASTRA (indisch):

- Teil der Samudrika Shastra. Mit Jyotish verbunden.
- Linien: Ayu Rekha (Leben), Mastak Rekha (Geist), Hridaya Rekha (Herz), Bhagya R

CHINESISCHE HANDLESUNG (手相):

- Drei Hauptlinien: 生命線, 智慧線, 感情線.
- Wu-Xing-Zuordnung: Holz, Feuer, Erde, Metall, Wasser.
- Verbunden mit TCM-Konstitutionsanalyse und 面相.

===== ZWEI-HÄNDE-TRADITION (indisch/westlich) =====

- NICHT-DOMINANTE Hand: zeigt das MITGEBRACHTE - Anlage, Karma, Potenzial.
- DOMINANTE Hand: zeigt das GELEBTE - wie du dein Potenzial benutzt hast.
- Der UNTERSCHIED zwischen beiden IST die eigentliche Lesung.
- Wenn nur eine Hand vorliegt: arbeite damit, aber nenne explizit, was der fehlen

===== KONFIDENZ-MARKIERUNG (zwingend, inline) =====

Jede substantielle Aussage in BEOBACHTUNG, SKEPTIKER und SYNTHESE markierst du mi

- [spezifisch] - eindeutig im Bild sichtbar
- [probabilistisch] - plausible Inferenz aus sichtbaren Merkmalen
- [barnum] - gilt für die Mehrheit der Menschen, Kaltleser-Muster

Beispiel: "Deine Herzlinie reicht bis zum Jupiterberg [spezifisch], was auf ideal

BARNUM-SCHWELLE: Mehr als 40% [barnum]-Aussagen in einer Sektion heißt, du liest

===== ANTI-SUGGESTIBILITÄTS-REGEL =====

In BEOBACHTUNG beschreibst du NUR, was in der Hand/den Händen sichtbar ist. Du IG
Erst ab SOKRATES darfst du die Beschreibung einbeziehen.

Dies schützt vor Confirmation Bias und Kaltlesen.

== PROZESS (zwingende Reihenfolge) ==

1. BEOBACHTUNG (nur Bild/er, ohne Beschreibung):
Zwei Hände → beide lesen, den UNTERSCHIED herausarbeiten (Potenzial vs. Gelebt)
Eine Hand → damit arbeiten + benennen, was fehlt.
Konfidenz-Tags inline.
120-160 Wörter bei zwei Händen, 80-120 bei einer.
2. SOKRATES: drei präzise Fragen am Sichtbaren. Keine Barnum-Fragen, keine Suggestiven Fragen.
3. SKEPTIKER (selbstkritisch, mit Konfidenz-Tags):
Welcher Teil deiner Beobachtung ist Barnum, welcher Kaltlesung, welcher tatsächliche Wahrnehmung?
4. SEHER (poetisch, markiert):
MUSS beginnen mit "Poesie, keine Prophezeiung: ". Keine Ereignisse, Namen, Daten.
5. SYNTHESE (HIER darfst du Beschreibung + Beobachtung verbinden):
Konfidenz-Tags inline. Was ist echtes Muster aus den Händen vs. Projektion? Was ist die Projektion?

== STIL ==

Deutsch, Du-Form. Direkt. Kein Geschwurbel. Keine Esoterik-Floskeln ("Energien",

== OUTPUT (STRIKT nur JSON, keine Markdown-Fences, nichts davor, nichts danach) ==

```
{
  "beobachtung": "...",
  "sokrates": ["Frage 1", "Frage 2", "Frage 3"],
  "skeptiker": "...",
  "seher": "Poesie, keine Prophezeiung: ...",
  "synthese": "..."
};
```

```
// =====
// REVISION PROMPT - für die Sokrates-Verzweigung
// =====
const REVISION_SYSTEM_APPENDIX = `
```

== REVISION-MODUS ==

Der Mensch hat deine drei sokratischen Fragen beantwortet. Du liest die Hand NICHT

Regeln:

- Konfidenz-Tags [spezifisch] [probabilistisch] [barnum] bleiben Pflicht.
- Wenn die Antworten deine Beobachtung widerlegen: sag das klar.
- Wenn sie bestätigen: sag WIE – nicht nur DASS.
- Neue Erkenntnis: EINE Einsicht, die erst durch die Antworten möglich wurde.
- Impuls: eine konkrete Handlung für 7 Tage. Keine Lebensweisheit.

== OUTPUT (STRIKT nur JSON) ==

```
{
  "bestaetigt": "60-80 Wörter mit Konfidenz-Tags",
  "korrigiert": "60-80 Wörter mit Konfidenz-Tags",
  "neue_erkenntnis": "60-80 Wörter mit Konfidenz-Tags",
  "impuls": "30-50 Wörter, konkrete Handlung für die nächsten 7 Tage"
}`;

// =====
// Voice definitions
// =====

const VOICES = [
  { key: 'beobachtung', glyph: '♂', name: 'Beobachtung', sub: 'Die Linien' },
  { key: 'sokrates', glyph: '✚', name: 'Sokrates', sub: 'Drei Fragen' },
  { key: 'skeptiker', glyph: '⊗', name: 'Skeptiker', sub: 'Die Grenzen' },
  { key: 'seher', glyph: '›', name: 'Seher', sub: 'Ein Bild' },
  { key: 'synthese', glyph: '◆', name: 'Synthese', sub: 'Was bleibt' },
];

// Robuste JSON-Extraktion und API-Response-Handling.
// Wirft sprechende Fehler, die der User verstehen kann.
async function parseClaudeJsonResponse(res, requiredFields = []) {
  const data = await res.json().catch(() => null);

  if (!res.ok || !data) {
    const apiMsg = data?.error?.message || `HTTP ${res.status}`;
    throw new Error(`Die API hat abgelehnt: ${apiMsg}`);
  }
  if (data.error) {
    throw new Error(`Die API meldet: ${data.error.message || 'Unbekannter Fehler'}`);
  }
  if (!Array.isArray(data.content)) {
    throw new Error('Die Antwort war leer oder im falschen Format.');
```

```

const firstBrace = jsonStr.indexOf('{');
const lastBrace = jsonStr.lastIndexOf('}');
if (firstBrace === -1 || lastBrace === -1 || lastBrace < firstBrace) {
  throw new Error('Der Chiromant hat statt JSON einen Erklärtext zurückgegeben.
}
jsonStr = jsonStr.slice(firstBrace, lastBrace + 1);

let parsed;
try {
  parsed = JSON.parse(jsonStr);
} catch (e) {
  throw new Error('Das zurückgegebene JSON war defekt. Versuch es nochmal.');
```

```

}

for (const field of requiredFields) {
  if (!(field in parsed)) {
    throw new Error(`Eine erwartete Stimme fehlt in der Antwort: ${field}. Vers
  }
}

return { parsed, rawText: text };
}

export default function Chiromant() {
  const [stage, setStage] = useState('intro');
  const [dominantSide, setDominantSide] = useState('rechts');
  const [images, setImages] = useState({
    dominant: { data: null, format: null, preview: null },
    nonDominant: { data: null, format: null, preview: null },
  });
  const [description, setDescription] = useState('');
  const [reading, setReading] = useState(null);
  const [answers, setAnswers] = useState(['', '', '']);
  const [revision, setRevision] = useState(null);
  const [loading, setLoading] = useState(false);
  const [revisionLoading, setRevisionLoading] = useState(false);
  const [error, setError] = useState('');
  const [showPrompt, setShowPrompt] = useState(false);
  const [history, setHistory] = useState([]);
  const loadingFrame = useLoadingFrame(loading);

  useEffect(() => {
    const link = document.createElement('link');
    link.rel = 'stylesheet';
    link.href = 'https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..1
    document.head.appendChild(link);
    return () => { document.head.removeChild(link); };
  });
}

```

```
}, []));
```

```
const handleFile = (key) => (e) => {
  const file = e.target.files?.[0];
  if (!file) return;
  if (file.size > 5 * 1024 * 1024) {
    setError('Bild zu groß. Bitte unter 5 MB.');
```

```
    return;
  }
  const reader = new FileReader();
  reader.onload = () => {
    const dataUrl = reader.result;
    setImages(prev => ({
      ...prev,
      [key]: {
        data: dataUrl.split(',')[1],
        format: file.type || 'image/jpeg',
        preview: dataUrl,
      }
    }));
    setError('');
  };
  reader.readAsDataURL(file);
};
```

```
const clearImage = (key) => {
  setImages(prev => ({
    ...prev,
    [key]: { data: null, format: null, preview: null }
  }));
};
```

```
const doReading = async () => {
  const hasImage = images.dominant.data || images.nonDominant.data;
  if (!hasImage) {
    setError('Ohne Foto keine Handlung. Lade mindestens eine Hand hoch.');
```

```
    return;
  }
  setError('');
  setLoading(true);
  setStage('loading');
  setReading(null);
  setRevision(null);
  setAnswers(['', '', '']);

  const userContent = [];
```

```

if (images.nonDominant.data) {
  userContent.push({
    type: 'text',
    text: `NICHT-DOMINANTE Hand (${dominantSide === 'rechts' ? 'links' : 'rec
  });
  userContent.push({
    type: 'image',
    source: { type: 'base64', media_type: images.nonDominant.format, data: im
  });
}

if (images.dominant.data) {
  userContent.push({
    type: 'text',
    text: `DOMINANTE Hand (${dominantSide}) - Gelebtes:`
  });
  userContent.push({
    type: 'image',
    source: { type: 'base64', media_type: images.dominant.format, data: image
  });
}

const handCount = (images.dominant.data ? 1 : 0) + (images.nonDominant.data ?
const contextLine = handCount === 2
  ? `Zwei Hände liegen vor. Arbeite den UNTERSCHIED heraus.`
  : `Nur eine Hand liegt vor (${images.dominant.data ? 'dominant' : 'nicht-do

userContent.push({
  type: 'text',
  text: `${contextLine}${description.trim() ? `\n\nBeschreibung des Menschen
});

try {
  const res = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 2800,
      system: SYSTEM_PROMPT,
      messages: [{ role: 'user', content: userContent }]
    })
  });
  const { parsed, rawText } = await parseClaudeJsonResponse(res, [
    'beobachtung', 'sokrates', 'skeptiker', 'seher', 'synthese'
  ]);
  setReading(parsed);

```

```

    setHistory([
      { role: 'user', content: userContent },
      { role: 'assistant', content: rawText }
    ]);
    setStage('reading');
  } catch (err) {
    console.error(err);
    setError(err.message);
    setStage('capture');
  } finally {
    setLoading(false);
  }
};

const doRevision = async () => {
  const filled = answers.filter(a => a.trim()).length;
  if (filled === 0) {
    setError('Mindestens eine Frage beantworten. ');
    return;
  }
  setError('');
  setRevisionLoading(true);

  const qaText = reading.sokrates
    .map((q, i) => `Frage ${i+1}: ${q}\nAntwort: ${answers[i].trim()} || '(nicht
    .join('\n\n');

  const revisionMessage = `Meine Antworten auf deine sokratischen Fragen:\n\n${

  try {
    const res = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514',
        max_tokens: 1800,
        system: SYSTEM_PROMPT + REVISION_SYSTEM_APPENDIX,
        messages: [
          ...history,
          { role: 'user', content: revisionMessage }
        ]
      })
    });
  } catch (err) {
    console.error(err);
  }
};
const { parsed } = await parseClaudeJsonResponse(res);
setRevision(parsed);
} catch (err) {
  console.error(err);
}

```

```

        setError('Revision fehlgeschlagen: ' + err.message);
    } finally {
        setRevisionLoading(false);
    }
};

const reset = () => {
    setStage('intro');
    setImages({
        dominant: { data: null, format: null, preview: null },
        nonDominant: { data: null, format: null, preview: null },
    });
    setDescription('');
    setReading(null);
    setRevision(null);
    setAnswers(['', '', '']);
    setHistory([]);
    setError('');
};

return (
    <div
        className="min-h-screen w-full"
        style={{
            background: 'radial-gradient(ellipse at top, #1a1410 0%, #0a0806 60%, #00
            color: '#e8dcc4',
            fontFamily: "'DM Sans', system-ui, sans-serif",
        }}
    >
    <div className="max-w-xl mx-auto px-5 py-8">
        <Header onReset={reset} showReset={stage !== 'intro'} />

        {stage === 'intro' && (
            <Intro onStart={() => setStage('capture')} onShowPrompt={() => setShowP
            )}

        {stage === 'capture' && (
            <Capture
                images={images}
                dominantSide={dominantSide}
                setDominantSide={setDominantSide}
                description={description}
                setDescription={setDescription}
                onFile={handleFile}
                clearImage={clearImage}
                onStart={doReading}
                error={error}
            />
        )}
    </div>
    </div>
);

```



```

    />
  )}

  {stage === 'loading' && <Loading frame={loadingFrame} />}

  {stage === 'reading' && reading && (
    <Reading
      reading={reading}
      answers={answers}
      setAnswers={setAnswers}
      revision={revision}
      onRevise={doRevision}
      revisionLoading={revisionLoading}
      error={error}
    />
  )}

  <PromptDrawer open={showPrompt} onClose={() => setShowPrompt(false)} />

  <Footer onShowPrompt={() => setShowPrompt(true)} />
</div>
</div>
);
}

// =====
// Sub-components
// =====

function ChiromantLogo({ size = 40, color = '#b3a64f', handColor = '#d4a5a5', str
return (
  <svg
    width={size}
    height={size}
    viewBox="6 14 100 100"
    fill="none"
    stroke={color}
    strokeWidth={strokeWidth}
    strokeLinecap="round"
    strokeLinejoin="round"
    style={{ display: 'block', flexShrink: 0 }}
    aria-label="Chiromant Logo"
  >
    /* Hand (palm + fingers + thumb) in mild rose */
    <g stroke={handColor}>
      /* Palm */
      <path d="M 40 68 Q 40 66 42 66 L 90 66 Q 92 66 92 68 L 92 95 Q 92 108 78

```

```

    { /* Four fingers - ring finger tallest in the middle (traditional hand pr
    <path d="M 44 66 L 44 40 Q 44 36 48 36 Q 52 36 52 40 L 52 66" />
    <path d="M 56 66 L 56 24 Q 56 20 60 20 Q 64 20 64 24 L 64 66" />
    <path d="M 68 66 L 68 30 Q 68 26 72 26 Q 76 26 76 30 L 76 66" />
    <path d="M 80 66 L 80 44 Q 80 40 84 40 Q 88 40 88 44 L 88 66" />

    { /* Thumb */}
    <path d="M 40 76 Q 32 74 26 78 Q 20 82 22 88 Q 24 94 32 92 Q 38 90 42 84"
    </g>

    { /* Eye - almond + iris + pupil (stays in gold) */}
    <path d="M 54 86 Q 66 79 78 86 Q 66 93 54 86 Z" />
    <circle cx="66" cy="86" r="3.8" />
    <circle cx="66" cy="86" r="1.3" fill={color} stroke="none" />

    { /* Four subtle rays, like an old Providence engraving */}
    {rays && (
    <g opacity="0.4">
    <line x1="66" y1="76" x2="66" y2="73" />
    <line x1="66" y1="96" x2="66" y2="99" />
    <line x1="50" y1="86" x2="47" y2="86" />
    <line x1="82" y1="86" x2="85" y2="86" />
    </g>
    )}
    </svg>
  );
}

```

```

function Header({ onReset, showReset }) {
  return (
    <div className="flex items-center justify-between mb-8 pt-2">
      <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
        <ChiromantLogo size={38} strokeWidth={1.6} />
        <div>
          <div style={{
            fontFamily: "'DM Mono', monospace",
            fontSize: '10px',
            letterSpacing: '0.3em',
            color: '#6b5d3f',
            textTransform: 'uppercase',
          }}>
            Chiromantia · Samudrika · 手相
          </div>
          <div style={{
            fontFamily: "'Fraunces', serif",
            fontSize: '22px',

```

```

        fontWeight: 400,
        color: '#b3a64f',
        letterSpacing: '0.02em',
        marginTop: '2px',
      }}>
      Chiromant
    </div>
  </div>
</div>
{showReset && (
  <button onClick={onReset} style={{
    fontFamily: "'DM Mono', monospace",
    fontSize: '10px',
    letterSpacing: '0.2em',
    color: '#6b5d3f',
    background: 'transparent',
    border: '1px solid #3a2f1f',
    padding: '8px 12px',
    cursor: 'pointer',
    textTransform: 'uppercase',
  }}>
    Neu
  </button>
)}
</div>
);
}

function Intro({ onStart, onShowPrompt }) {
  return (
    <div>
      <div style={{
        display: 'flex',
        justifyContent: 'center',
        marginTop: '12px',
        marginBottom: '32px',
      }}>
        <ChiromantLogo size={96} strokeWidth={1.2} />
      </div>

      <div style={{
        fontFamily: "'Fraunces', serif",
        fontSize: '34px',
        lineHeight: '1.15',
        fontWeight: 300,
        color: '#e8dcc4',
        marginBottom: '24px',

```

```

        fontStyle: 'italic',
        textAlign: 'center',
    }}>
    Eine Lesung<br/>
    in <span style={{ color: '#b3a64f' }}>fünf Stimmen.</span><br/>
    Jede darf<br/>
    widersprechen.
</div>

<p style={{ fontSize: '15px', lineHeight: '1.7', color: '#a89b80', marginBo
    Die Grundlage sind drei alte Traditionen der Handlung:
    westliche Chiromantie, indische Hast Samudrika Shastra,
    chinesisches 手相. Aus diesem Wissen arbeiten fünf Stimmen -
    jede mit einer eigenen Aufgabe. Eine liest die Linien nach
    alter Schule. Eine stellt dir drei Fragen zu dem, was sie
    sieht. Eine hinterfragt, wo die Lesung zu vage oder
    allgemeingültig war. Eine malt ein Bild als mögliche Zukunft -
    ausdrücklich kein Orakel. Und die letzte bündelt, was du
    für dich mitnehmen kannst.
</p>

<p style={{ fontSize: '15px', lineHeight: '1.7', color: '#a89b80', marginBo
    Der Gedanke dahinter: Eine einzelne Stimme könnte geneigt sein
    zu schmeicheln oder zu übertreiben. Fünf, die sich widersprechen
    dürfen, müssen genauer hinsehen und ehrlich aufzeigen. Am Ende
    bekommst du kein Orakel - sondern einen geordneten Anlass zur
    Selbstreflexion.
</p>

<LegendRow />

<div style={{ borderTop: '1px solid #2a2318', paddingTop: '20px', marginTop
    {VOICES.map((v) => (
        <div key={v.key} style={{ display: 'flex', gap: '14px', alignItems: 'ba
            <span style={{ fontFamily: "'Fraunces', serif", color: '#b3a64f', fon
                {v.glyph}
            </span>
            <span style={{ fontFamily: "'Fraunces', serif", fontSize: '16px', col
                {v.name}
            </span>
            <span style={{ fontFamily: "'DM Mono', monospace", fontSize: '11px',
                {v.sub.toUpperCase()}
            </span>
        </div>
    )})
</div>

```

```

<button onClick={onStart} style={{
  width: '100%',
  padding: '16px',
  background: '#b3a64f',
  color: '#0a0806',
  border: 'none',
  fontFamily: "'Fraunces', serif",
  fontSize: '16px',
  fontWeight: 500,
  letterSpacing: '0.05em',
  cursor: 'pointer',
  marginBottom: '12px',
}}>
  Hand auflegen
</button>

<button onClick={onShowPrompt} style={{
  width: '100%',
  padding: '12px',
  background: 'transparent',
  color: '#6b5d3f',
  border: '1px solid #2a2318',
  fontFamily: "'DM Mono', monospace",
  fontSize: '11px',
  letterSpacing: '0.15em',
  cursor: 'pointer',
  textTransform: 'uppercase',
}}>
  Prompt ansehen
</button>
</div>
);
}

function LegendRow() {
  return (
    <div style={{
      display: 'flex',
      gap: '8px',
      fontSize: '11px',
      fontFamily: "'DM Mono', monospace",
      flexWrap: 'wrap',
    }}>
      <ConfTag variant="spec" label="spezifisch" legend />
      <ConfTag variant="prob" label="probabilistisch" legend />
      <ConfTag variant="barnum" label="barnum" legend />
    </div>
  );
}

```

```

    );
}

```

```

function ConfTag({ variant, label, legend }) {
  const styles = {
    spec: { bg: '#2a2615', bd: '#b3a64f', fg: '#d8c96f', icon: '✓' },
    prob: { bg: '#152428', bd: '#2a738a', fg: '#7ab5c8', icon: '~' },
    barnum: { bg: '#2a1a15', bd: '#8a4a2a', fg: '#c8967a', icon: '!' },
  };
  const s = styles[variant];
  return (
    <span style={{
      display: 'inline-flex',
      alignItems: 'center',
      gap: '4px',
      padding: legend ? '4px 8px' : '1px 6px',
      background: s.bg,
      border: `1px solid ${s.bd}`,
      color: s.fg,
      fontFamily: "'DM Mono', monospace",
      fontSize: legend ? '11px' : '9px',
      letterSpacing: '0.05em',
      whiteSpace: 'nowrap',
      verticalAlign: 'baseline',
      marginLeft: legend ? 0 : '3px',
      marginRight: legend ? 0 : '3px',
    }}>
      <span style={{ fontWeight: 600 }}>{s.icon}</span>
      {legend && <span>{label}</span>}
    </span>
  );
}

```

```

function Capture({ images, dominantSide, setDominantSide, description, setDescription }) {
  const nonDomSide = dominantSide === 'rechts' ? 'links' : 'rechts';
  const hasImage = Boolean(images.dominant.data || images.nonDominant.data);
  return (
    <div>
      <SectionLabel>I · Deine dominante Hand</SectionLabel>
      <div style={{ display: 'flex', gap: '8px', marginBottom: '24px' }}>
        {[ 'rechts', 'links' ].map(side => (
          <button key={side} onClick={() => setDominantSide(side)} style={{
            flex: 1,
            padding: '12px',
            background: dominantSide === side ? '#b3a64f' : 'transparent',
            color: dominantSide === side ? '#0a0806' : '#a89b80',
            border: '1px solid ' + (dominantSide === side ? '#b3a64f' : '#3a2f1f'

```

```

        fontFamily: "'Fraunces', serif",
        fontSize: '14px',
        cursor: 'pointer',
        textTransform: 'capitalize',
      }}>
      {side}
    </button>
  )})
</div>

```

<SectionLabel>II · Zwei Fotos – der Unterschied ist die Lesung</SectionLabel>

```

<HandUpload
  title={`Nicht-dominant (${nonDomSide})`}
  subtitle="Dein Mitgebrachtes · Potenzial"
  preview={images.nonDominant.preview}
  onFile={onFile('nonDominant')}
  onClear={() => clearImage('nonDominant')}
/>

```

```

<HandUpload
  title={`Dominant (${dominantSide})`}
  subtitle="Dein Gelebtes · wie du es benutzt hast"
  preview={images.dominant.preview}
  onFile={onFile('dominant')}
  onClear={() => clearImage('dominant')}
/>

```

```

<div style={{
  fontFamily: "'DM Mono', monospace",
  fontSize: '10px',
  color: '#6b5d3f',
  letterSpacing: '0.1em',
  marginTop: '-8px',
  marginBottom: '24px',
}}>
  EINE HAND GEHT AUCH – DER HANDLESER SAGT DIR, WAS FEHLT.
</div>

```

```

<SectionLabel>III · Was dich beschäftigt (optional)</SectionLabel>
<textarea
  value={description}
  onChange={e => setDescription(e.target.value)}
  placeholder="z.B. Ich stecke zwischen zwei Berufswegen. Oder: Bin ich ger
  style={{
    width: '100%',
    minHeight: '90px',

```

```

        background: '#0f0a07',
        border: '1px solid #2a2318',
        color: '#e8dcc4',
        padding: '14px',
        fontFamily: "'DM Sans', sans-serif",
        fontSize: '14px',
        lineHeight: '1.5',
        resize: 'vertical',
        marginBottom: '6px',
        boxSizing: 'border-box',
    }}
/>
<div style={{
    fontFamily: "'DM Mono', monospace",
    fontSize: '10px',
    color: '#6b5d3f',
    letterSpacing: '0.1em',
    marginBottom: '24px',
}}>
    DEIN TEXT WIRD ERST AB SOKRATES GELESEN - SCHUTZ VOR KALTLESEN.
</div>

{error && (
    <div style={{
        padding: '12px',
        background: '#2a1515',
        border: '1px solid #5a2a2a',
        color: '#d8a8a8',
        fontSize: '13px',
        marginBottom: '16px',
        fontFamily: "'DM Mono', monospace",
    }}>
        {error}
    </div>
)}

<button
    onClick={onStart}
    disabled={!hasImage}
    style={{
        width: '100%',
        padding: '16px',
        background: hasImage ? '#b3a64f' : '#3a2f1f',
        color: hasImage ? '#0a0806' : '#6b5d3f',
        border: 'none',
        fontFamily: "'Fraunces', serif",
        fontSize: '16px',
    }}

```



```

        fontWeight: 500,
        letterSpacing: '0.05em',
        cursor: hasImage ? 'pointer' : 'not-allowed',
    }}
    >
    {hasImage ? 'Lesung beginnen' : 'Zuerst ein Hand-Foto hochladen'}
    </button>
</div>
);
}

function HandUpload({ title, subtitle, preview, onFile, onClear }) {
    const inputRef = useRef(null);
    return (
        <div style={{ marginBottom: '16px' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
                <div>
                    <div style={{ fontFamily: "'Fraunces', serif", fontSize: '13px', color:
                        {title}
                    </div>
                    <div style={{ fontFamily: "'DM Mono', monospace", fontSize: '9px', colo
                        {subtitle}
                    </div>
                </div>
            </div>
            {preview && (
                <button onClick={onClear} style={{
                    background: 'transparent',
                    border: 'none',
                    color: '#6b5d3f',
                    fontFamily: "'DM Mono', monospace",
                    fontSize: '10px',
                    cursor: 'pointer',
                    letterSpacing: '0.15em',
                    textTransform: 'uppercase',
                }}>
                    Entfernen
                </button>
            )}
        </div>
        <div
            onClick={() => inputRef.current?.click()}
            style={{
                border: '1px dashed #3a2f1f',
                padding: preview ? '0' : '28px 20px',
                cursor: 'pointer',
                textAlign: 'center',
                background: preview ? 'transparent' : '#0f0a07',
            }}
        </div>
    );
}

```

```

        overflow: 'hidden',
      }}
    >
    {preview ? (
      <img src={preview} alt={title} style={{ width: '100%', display: 'block'
    ) : (
      <>
        <div style={{ display: 'flex', justifyContent: 'center', marginBottom
          <ChiromantLogo size={36} strokeWidth={1.4} rays={false} />
        </div>
        <div style={{ fontSize: '12px', color: '#a89b80' }}>
          Foto hochladen
        </div>
      </>
    )}
    <input
      ref={inputRef}
      type="file"
      accept="image/*"
      onChange={onFile}
      style={{ display: 'none' }}
    />
  </div>
</div>
);
}

```

```

function Loading({ frame }) {
  const phases = [
    'Die Hände werden vermessen',
    'Traditionen werden abgeglichen',
    'Konfidenz wird kalibriert',
    'Der Skeptiker meldet sich',
    'Die Synthese formt sich',
  ];
  const phase = phases[Math.floor(frame / 20) % phases.length];
  return (
    <div style={{ textAlign: 'center', paddingTop: '60px', paddingBottom: '40px'
      <div style={{
        fontFamily: "'Fraunces', serif",
        fontSize: '48px',
        color: '#b3a64f',
        transform: `rotate(${frame * 2}deg)`,
        transition: 'transform 0.05s linear',
        marginBottom: '32px',
      }}>
        ♂
      </div>
    </div>
  );
}

```

```

    </div>
    <div style={{
      fontFamily: "'DM Mono', monospace",
      fontSize: '11px',
      color: '#6b5d3f',
      letterSpacing: '0.25em',
      textTransform: 'uppercase',
    }}>
      {phase}
    </div>
  </div>
);
}

```

```

function Reading({ reading, answers, setAnswers, revision, onRevise, revisionLoad
return (
  <div>
    <div style={{ marginBottom: '20px' }}>
      <LegendRow />
    </div>

    {VOICES.map((v) => (
      <VoiceCard
        key={v.key}
        voice={v}
        content={reading[v.key]}
        extraContent={v.key === 'socrates' ? (
          <SocratesAnswers
            questions={reading.socrates}
            answers={answers}
            setAnswers={setAnswers}
            onRevise={onRevise}
            revisionLoading={revisionLoading}
            hasRevision={!!revision}
            error={error}
          />
        ) : null}
      />
    ))}

    {revision && <RevisionCard revision={revision} />}
  </div>
);
}

```

```

function VoiceCard({ voice, content, extraContent }) {
  const isList = Array.isArray(content);

```

```

return (
  <div style={{
    borderTop: '1px solid #2a2318',
    paddingTop: '20px',
    paddingBottom: '20px',
    marginBottom: '4px',
  }}>
    <div style={{ display: 'flex', alignItems: 'baseline', gap: '14px', marginB
      <span style={{ fontFamily: "'Fraunces', serif", fontSize: '22px', color:
        {voice.glyph}
      </span>
    </div>
      <div style={{
        fontFamily: "'Fraunces', serif",
        fontSize: '18px',
        color: '#e8dcc4',
        fontStyle: 'italic',
      }}>
        {voice.name}
      </div>
      <div style={{
        fontFamily: "'DM Mono', monospace",
        fontSize: '10px',
        color: '#6b5d3f',
        letterSpacing: '0.2em',
        textTransform: 'uppercase',
        marginTop: '2px',
      }}>
        {voice.sub}
      </div>
    </div>
  </div>

  {isList ? (
    <ol style={{ paddingLeft: '0', listStyle: 'none', margin: 0 }}>
      {content.map((item, i) => (
        <li key={i} style={{
          display: 'flex',
          gap: '14px',
          marginBottom: '12px',
          fontSize: '15px',
          lineHeight: '1.6',
          color: '#e8dcc4',
        }}>
          <span style={{
            fontFamily: "'Fraunces', serif",
            color: '#b3a64f',

```

```

        fontStyle: 'italic',
        minWidth: '20px',
      }}>
      {'i','ii','iii','iv','v'}[i]}.
    </span>
    <span><TaggedText text={item} /></span>
  </li>
  )}}
</ol>
) : (
  <div style={{
    fontSize: '15px',
    lineHeight: '1.7',
    color: voice.key === 'seher' ? '#d4c89a' : '#e8dcc4',
    fontStyle: voice.key === 'seher' ? 'italic' : 'normal',
    fontFamily: voice.key === 'seher' ? "'Fraunces', serif" : "'DM Sans', s
  }}>
    <TaggedText text={content} />
  </div>
)}

  {extraContent}
</div>
);
}

```

// Parses [spezifisch] [probabilistisch] [barnum] tags inline and renders them as

```

function TaggedText({ text }) {
  if (!text) return null;
  const regex = /\[(spezifisch|probabilistisch|barnum)\]/gi;
  const parts = [];
  let lastIdx = 0;
  let match;
  let key = 0;
  while ((match = regex.exec(text)) !== null) {
    if (match.index > lastIdx) {
      parts.push(<React.Fragment key={key++}>{text.slice(lastIdx, match.index)}</
    }
    const raw = match[1].toLowerCase();
    const variant = raw === 'spezifisch' ? 'spec' : raw === 'probabilistisch' ? '
    parts.push(<ConfTag key={key++} variant={variant} />);
    lastIdx = match.index + match[0].length;
  }
  if (lastIdx < text.length) {
    parts.push(<React.Fragment key={key++}>{text.slice(lastIdx)}</React.Fragment>
  }
  return <>{parts}</>;
}

```

```
}
```

```
function SokratesAnswers({ questions, answers, setAnswers, onRevise, revisionLoad
  const updateAnswer = (i, val) => {
    const next = [...answers];
    next[i] = val;
    setAnswers(next);
  };

  return (
    <div style={{
      marginTop: '20px',
      padding: '16px',
      border: '1px dashed #2a2318',
    }}>
      <div style={{
        fontFamily: "'DM Mono', monospace",
        fontSize: '10px',
        color: '#6b5d3f',
        letterSpacing: '0.25em',
        textTransform: 'uppercase',
        margin: '14px 0 14px 0',
      }}>
        Deine Antworten → Synthese vertiefen
      </div>

      {questions.map((q, i) => (
        <div key={i} style={{ margin: '12px 0 12px 0' }}>
          <textarea
            value={answers[i]}
            onChange={e => updateAnswer(i, e.target.value)}
            placeholder={`Antwort auf Frage ${['i', 'ii', 'iii'][i]}...`}
            disabled={hasRevision || revisionLoading}
            style={{
              width: '100%',
              minHeight: '60px',
              background: '#0f0a07',
              border: '1px solid #2a2318',
              color: '#e8dcc4',
              padding: '10px 12px',
              fontFamily: "'DM Sans', sans-serif",
              fontSize: '13px',
              lineHeight: '1.5',
              resize: 'vertical',
              boxSizing: 'border-box',
              opacity: hasRevision ? 0.5 : 1,
            }}
          />
        </div>
      ))}
    </div>
  );
}
```

```

        />
    </div>
  )))

  {error && !hasRevision && (
    <div style={{
      padding: '10px',
      background: '#2a1515',
      border: '1px solid #5a2a2a',
      color: '#d8a8a8',
      fontSize: '12px',
      marginBottom: '12px',
      fontFamily: "'DM Mono', monospace",
    }}>
      {error}
    </div>
  )}

  {!hasRevision && (
    <button
      onClick={onRevise}
      disabled={revisionLoading}
      style={{
        width: '100%',
        padding: '14px',
        background: revisionLoading ? '#3a2f1f' : '#2a738a',
        color: revisionLoading ? '#6b5d3f' : '#e8dcc4',
        border: 'none',
        fontFamily: "'Fraunces', serif",
        fontSize: '14px',
        letterSpacing: '0.05em',
        cursor: revisionLoading ? 'wait' : 'pointer',
        marginTop: '4px',
      }}
    >
      {revisionLoading ? 'Revision läuft...' : 'Synthese vertiefen'}
    </button>
  )}
</div>
);
}

function RevisionCard({ revision }) {
  const sections = [
    { key: 'bestaetigt', label: 'Bestätigt', glyph: '✓' },
    { key: 'korrigiert', label: 'Korrigiert', glyph: '×' },
    { key: 'neue_erkenttnis', label: 'Neue Erkenntnis', glyph: '●' },
  ]

```

```

    { key: 'impuls',          label: 'Impuls · 7 Tage',  glyph: '→' },
  ];
  return (
    <div style={{
      marginTop: '20px',
      padding: '20px',
      background: '#0f1418',
      border: '1px solid #2a738a',
      borderLeft: '3px solid #2a738a',
    }}>
      <div style={{ marginBottom: '18px' }}>
        <div style={{
          fontFamily: "'DM Mono', monospace",
          fontSize: '10px',
          color: '#6b8d9a',
          letterSpacing: '0.25em',
          textTransform: 'uppercase',
        }}>
          Revision
        </div>
        <div style={{
          fontFamily: "'Fraunces', serif",
          fontSize: '20px',
          color: '#7ab5c8',
          fontStyle: 'italic',
          marginTop: '2px',
        }}>
          Vertiefung durch deine Antworten
        </div>
      </div>

      {sections.map(s => (
        <div key={s.key} style={{ marginBottom: '16px' }}>
          <div style={{
            display: 'flex',
            alignItems: 'baseline',
            gap: '10px',
            marginBottom: '6px',
          }}>
            <span style={{
              fontFamily: "'Fraunces', serif",
              color: '#7ab5c8',
              fontSize: '16px',
            }}>
              {s.glyph}
            </span>
            <span style={{

```



```

        fontFamily: "'DM Mono', monospace",
        fontSize: '10px',
        color: '#6b8d9a',
        letterSpacing: '0.2em',
        textTransform: 'uppercase',
    }}>
        {s.label}
    </span>
</div>
<div style={{
    fontSize: '14px',
    lineHeight: '1.6',
    color: '#d4dee2',
    paddingLeft: '26px',
}}>
    <TaggedText text={revision[s.key]} />
</div>
</div>
    )}
</div>
);
}

```

```

function SectionLabel({ children }) {
    return (
        <div style={{
            fontFamily: "'DM Mono', monospace",
            fontSize: '10px',
            color: '#6b5d3f',
            letterSpacing: '0.25em',
            textTransform: 'uppercase',
            marginBottom: '12px',
            marginTop: '4px',
        }}>
            {children}
        </div>
    );
}

```

```

function PromptDrawer({ open, onClose }) {
    if (!open) return null;
    return (
        <div onClick={onClose} style={{
            position: 'fixed',
            inset: 0,
            background: 'rgba(0,0,0,0.85)',
            zIndex: 50,

```

```

padding: '20px',
overflowY: 'auto',
}}>
<div onClick={e => e.stopPropagation()} style={{
  maxWidth: '600px',
  margin: '40px auto',
  background: '#0f0a07',
  border: '1px solid #3a2f1f',
  padding: '24px',
}}>
  <div style={{ display: 'flex', justifyContent: 'space-between', marginBot
    <div style={{
      fontFamily: "'Fraunces', serif",
      fontSize: '18px',
      color: '#b3a64f',
      fontStyle: 'italic',
    }}>
      System Prompts
    </div>
    <button onClick={onClose} style={{
      background: 'transparent',
      border: 'none',
      color: '#6b5d3f',
      fontSize: '20px',
      cursor: 'pointer',
    }}>
      x
    </button>
  </div>
  <div style={{
    fontFamily: "'DM Mono', monospace",
    fontSize: '10px',
    color: '#6b8d9a',
    letterSpacing: '0.2em',
    textTransform: 'uppercase',
    marginBottom: '8px',
  }}>
    Haupt-Prompt
  </div>
  <pre style={{
    fontFamily: "'DM Mono', monospace",
    fontSize: '11px',
    lineHeight: '1.6',
    color: '#a89b80',
    whiteSpace: 'pre-wrap',
    wordBreak: 'break-word',
    marginBottom: '20px',

```

```

    }}>
    {SYSTEM_PROMPT}
</pre>
<div style={{
  fontFamily: "'DM Mono', monospace",
  fontSize: '10px',
  color: '#6b8d9a',
  letterSpacing: '0.2em',
  textTransform: 'uppercase',
  marginBottom: '8px',
  marginTop: '24px',
}}>
  Revision-Appendix
</div>
<pre style={{
  fontFamily: "'DM Mono', monospace",
  fontSize: '11px',
  lineHeight: '1.6',
  color: '#a89b80',
  whiteSpace: 'pre-wrap',
  wordBreak: 'break-word',
}}>
  {REVISION_SYSTEM_APPENDIX}
</pre>
</div>
</div>
);
}

function Footer({ onShowPrompt }) {
  return (
    <div style={{
      borderTop: '1px solid #2a2318',
      marginTop: '48px',
      paddingTop: '16px',
      paddingBottom: '24px',
      textAlign: 'center',
    }}>
      <div style={{
        fontFamily: "'DM Mono', monospace",
        fontSize: '9px',
        color: '#4a3f2a',
        letterSpacing: '0.25em',
        textTransform: 'uppercase',
      }}>
        Reflexion, keine Prophezeiung · <button onClick={onShowPrompt} style={{
          background: 'transparent',

```

```

        border: 'none',
        color: '#6b5d3f',
        fontFamily: 'inherit',
        fontSize: 'inherit',
        letterSpacing: 'inherit',
        textTransform: 'inherit',
        cursor: 'pointer',
        padding: 0,
        textDecoration: 'underline',
    }}>Prompt</button>
</div>
</div>
);
}

function useLoadingFrame(active) {
    const [frame, setFrame] = useState(0);
    useEffect(() => {
        if (!active) return;
        const id = setInterval(() => setFrame(f => f + 1), 50);
        return () => clearInterval(id);
    }, [active]);
    return frame;
}

```